

Описание кодовых ячеек `artefact.ipynb`

Человекочитаемое пояснение к ноутбуку PassLLM / OpenEvolve / vLLM

Общая идея ноутбука

Ноутбук решает задачу `targeted password guessing`: по старому паролю нужно сгенерировать наиболее вероятные варианты нового пароля. Маленькая модель PassLLM на базе Qwen/Qwen2.5-0.5B-Instruct вместе с LoRA-адаптером генерирует кандидатов, а OpenEvolve при помощи qwen3:8b постепенно переписывает системный промпт, чтобы эти кандидаты становились лучше.

В этом отчёте оставлены только исполняемые ячейки. Markdown-блоки из ноутбука намеренно не описываются: они не запускают код и не меняют состояние эксперимента. Нумерация ниже идёт именно по кодовым ячейкам, поэтому первая ячейка в отчёте — это первая исполняемая ячейка ноутбука.

Подробное описание кодовых ячеек

- Кодовая ячейка 1. Установка библиотек и системной зависимости.** Первая ячейка устанавливает необходимые библиотеки, которые дальше используются почти во всём ноутбуке. Через `pip` ставятся `openevolve`, `transformers`, `peft`, `accelerate`, `pyyaml`, `flask`, `psutil` и `setproctitle`. Это нужно, чтобы можно было запускать OpenEvolve, загружать модели HuggingFace, подключать LoRA-адаптеры и контролировать процессы.

Также при наличии Linux ячейка устанавливает `zstd` через `apt-get install zstd`. Это сделано заранее, чтобы в дальнейшем нормально установить и запустить Ollama: без системных утилит установка Ollama или загрузка её артефактов может упасть не из-за Python-кода, а из-за отсутствующей зависимости.

Важные параметры и действия:

- `pip install -q` — тихая установка Python-зависимостей;
- `sys.platform == 'Linux'` — проверка, что окружение Linux;
- `apt-get install zstd` — системная подготовка под Ollama.

- Кодовая ячейка 2. Настройка путей до файлов и окружения.** Вторая ячейка решает проблему путей до нужных файлов. Она проверяет, запускается ли ноутбук на Kaggle, и в зависимости от этого выставляет путь к папке соревнования, к `train.json` и к LoRA-адаптеру `126_csdn_disQwen0.5B`. Благодаря этому один и тот же код можно запускать и в Kaggle, и локально: если Kaggle-папка есть, используются Kaggle-пути, иначе берётся переменная окружения или текущая директория.

Эта же ячейка создаёт рабочую папку `targeted_evolution_exp`, куда дальше будут записываться конфиги, `evaluator`, начальный промпт, лучший промпт и результаты OpenEvolve. Также она выставляет переменные окружения, чтобы следующие ячейки не передавали пути вручную каждый раз.

Отдельно здесь настраивается `multiprocessing`. Если способ запуска процессов ещё не задан, выбирается `spawn`. Это полезно для ноутбуков с GPU, потому что дочерние процессы стартуют более чисто и не наследуют лишнее состояние от `kernel`-процесса.

Важные параметры:

- `COMPETITION` — корневая папка данных соревнования;
- `PATH_TRAIN` — путь к `train.json`;
- `PATH_ADAPTER_126_CSDN` — путь к LoRA-адаптеру;
- `BASE_MODEL_PASSLLM` — базовая модель PassLLM;
- `EXPERIMENT_DIR` — рабочая директория эксперимента;
- `CUDA_VISIBLE_DEVICES=0` — выбор первой CUDA-видеокарты;
- `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` — более гибкое управление CUDA-памятью.

3. **Кодовая ячейка 3. Выбор устройства для LLM-генерации.** Третья ячейка устанавливает `device`, на котором будет выполняться LLM-генерация. Сначала проверяется `cuda`, затем `mps`, и если ускорителей нет, используется `cpu`. Результат сохраняется в `os.environ["DEVICE"]`, чтобы `evaluator` и другие части кода могли одинаково понимать, куда загружать модель.

В этой же ячейке задаётся `HF_TOKEN`. Он нужен для доступа к моделям HuggingFace, если среда требует авторизацию. В отчёте сам токен не приводится, потому что такие значения лучше хранить в секретах Kaggle или в приватных переменных окружения, а не оставлять в публичном коде.

Важные параметры:

- `torch.cuda.is_available()` — проверка NVIDIA GPU;
- `torch.backends.mps.is_available()` — проверка Apple Silicon GPU;
- `DEVICE` — итоговый выбранный backend: `cuda`, `mps` или `cpu`.

4. **Кодовая ячейка 4. Проверка размера данных.** Четвёртая ячейка просто пишет размер данных, на которых дальше проверяется качество промпта. Она читает `train.json`, приводит данные к списку, если JSON почему-то загружен как один словарь, и печатает количество объектов.

Ещё ячейка выводит первый пример из датасета. Это простая, но полезная проверка: сразу видно, что файл найден, JSON прочитан корректно, а структура данных действительно содержит старый пароль и целевой новый пароль.

5. **Кодовая ячейка 5. Параметры OpenEvolve и системный промпт для qwen3:8b.** Пятая ячейка задаёт параметры OpenEvolve и системный промпт для `qwen3:8b`. Здесь важно понимать, что `qwen3:8b` не угадывает пароли напрямую. Она выступает как модель, которая переписывает промпт для маленькой PassLLM-модели. Потом `evaluator` проверяет, стал ли новый промпт лучше.

В этой ячейке задаются основные настройки эволюции: количество итераций, размер популяции, число островов, период миграции, температура мутаций и число параллельных оценок. Также задаётся `NUM_GUESSES=100`: то есть для одного старого пароля модель должна выдать до ста кандидатов. Это число потом используется и в `evaluator`-е, и при генерации `submission`.

Системный промпт для `qwen3:8b` написан не абстрактно, а под конкретную задачу. В него включены наблюдения по датасету: как часто пароль не меняется, как часто добавляется суффикс или префикс, как меняется длина, какие есть популярные числовые добавки. Это нужно, чтобы OpenEvolve улучшал промпт не вслепую, а с учётом реального распределения данных.

Важные параметры:

- `OPENEVOLVE_MAX_ITERATIONS=60` — число итераций эволюции;
- `CHECKPOINT_INTERVAL=5` — как часто сохранять промежуточное состояние;
- `MUTATION_MODEL=qwen3:8b` — модель, которая переписывает промпт;
- `POPULATION_SIZE=16` — размер популяции;
- `NUM_ISLANDS=3` — число островов в эволюционном поиске;
- `MIGRATION_INTERVAL=8` — как часто программы мигрируют между островами;
- `MUTATION_TEMPERATURE=0.7` — насколько разнообразными будут изменения промпта;
- `PARALLEL_EVALUATIONS=2` — сколько оценок можно запускать параллельно;
- `KEEP_OLLAMA_WEIGHTS_LOADED=True` — попытка держать веса Ollama в памяти, чтобы не загружать их заново.

6. **Кодовая ячейка 6. Подключение vLLM.** Шестая ячейка подключает vLLM, чтобы более плотно задействовать ресурсы GPU. Обычная генерация через transformers проще, но при большом числе запросов может использовать видеокарту не полностью. vLLM нужен для более плотного batching и более быстрой генерации большого числа парольных кандидатов.

Главный переключатель здесь — `USE_VLLM=True`. Если поставить `False`, можно вернуться к обычному evaluator-у. Также ячейка задаёт порт, имя модели внутри vLLM и ограничения по памяти, длине контекста и числу параллельных sequence.

Важные параметры:

- `USE_VLLM=True` — включение vLLM-режима;
- `VLLM_GPU_MEMORY_FRACTION=0.60` — какую долю GPU-памяти может использовать vLLM;
- `VLLM_MAX_MODEL_LEN=5000` — максимальная длина контекста;
- `VLLM_MAX_NUM_SEQS=64` — максимальное число параллельных sequence;
- `VLLM_BATCH_SIZE=16` — размер батча на стороне ноутбука;
- `VLLM_PORT=8000` — порт локального API;
- `VLLM_MODEL_NAME=passllm` — имя модели с LoRA-адаптером внутри vLLM.

7. **Кодовая ячейка 7. Запуск vLLM.** Седьмая ячейка запускает vLLM. Сначала она проверяет, установлен ли пакет `vllm`. Если пакет не найден, ячейка устанавливает `vllm[cuda]>=0.4.0`. После этого собирается команда запуска OpenAI-compatible API-сервера через `python -m vllm.entrypoints.openai.api_server`.

Важная часть этой ячейки — подключение LoRA-адаптера. Если путь к `126_csdn_disQwen0.5B` существует, в команду добавляется LoRA-модуль с именем `passllm`. Поэтому дальше evaluator может обращаться к модели как к `passllm`, но фактически использовать базовую Qwen-модель вместе с адаптером соревнования.

После запуска процесс не используется сразу. Ячейка ждёт готовности endpoint-a `/v1/models`, чтобы следующие шаги не начали отправлять запросы в сервер, который ещё не поднялся.

Важные параметры запуска:

- `--model` — базовая модель PassLLM;
- `--enable-lora` — включение LoRA;
- `--lora-modules passllm=...` — подключение адаптера;
- `--gpu-memory-utilization` — лимит использования VRAM;
- `--max-model-len` — ограничение контекста;
- `--max-num-seqs` — ограничение параллелизма;
- `--no-enable-log-requests` — отключение лишних логов запросов.

8. **Кодовая ячейка 8. Создание начального targeted-промпта.** Восьмая ячейка создаёт начальный системный промпт для PassLLM и сохраняет его в файл `initial_prompt.txt`. Это стартовая точка для OpenEvolve: дальше эволюция будет улучшать именно этот текст.

Промпт объясняет маленькой модели, что нужно преобразовать OLD password в наиболее вероятный NEW password. Очень жёстко прописаны правила вывода: писать ровно один пароль, без объяснений, без списков, без кавычек, без markdown и без лишних пробелов. Это важно, потому что evaluator потом сравнивает строку с правильным паролем, и лишний текст ломал бы проверку.

Также в промпте задан порядок предпочтений: сначала старый пароль без изменений, потом короткий суффикс, потом короткий префикс, потом небольшая локальная правка, и только после этого полный reset на популярный пароль. Отдельно описана ситуация, когда decoder принудительно задаёт длину вывода: модель должна выбирать самый вероятный пароль именно нужной длины.

Важный параметр: NUM_GUESSES берётся из переменной окружения PASSLLM_NUM_GUESSES, поэтому промпт согласован с числом кандидатов, заданным раньше.

9. **Кодовая ячейка 9. Выбор исходного кода evaluator-a.** Девятая ячейка выбирает, какой evaluator использовать. Если ранее был включён vLLM и существует файл `evaluator_adaptive.py`, ячейка читает его исходный код и выбирает его как evaluator. Если такого файла нет, она оставляет выбор пустым, чтобы следующая ячейка создала обычный evaluator.

По смыслу эта ячейка делает развилку между быстрым vLLM-backed evaluator-ом и стандартным evaluator-ом. Оба нужны для одного и того же: взять очередной промпт от OpenEvolve, сгенерировать кандидатов для train-примеров и вернуть метрику качества. Разница только в том, через какой backend выполняется генерация.

10. **Кодовая ячейка 10. Генерация файла evaluator.py.** Десятая ячейка создаёт файл `evaluator.py`. Это одна из самых важных ячеек, потому что именно этот evaluator OpenEvolve запускает на каждой итерации. Он получает новый промпт, проверяет его на части обучающих данных и возвращает score.

Внутри evaluator-a есть логика загрузки модели: загружается токенизатор, базовая модель Qwen/Qwen2.5-0.5B-Instruct и LoRA-адаптер. Если адаптер не найден локально, код пытается скачать или распаковать нужные веса. Это нужно, чтобы evaluator мог работать как на Kaggle, так и в другом окружении.

Главная функция генерации — `guess_one`. Она принимает старый пароль и системный промпт, запускает `generation` и собирает уникальные кандидаты. Далее функция `evaluate` прогоняет промпт на случайной подвыборке `train.json` и считает `cracked_rate`: долю случаев, где правильный новый пароль попал в список кандидатов. Этот `cracked_rate` и используется как основной `combined_score`.

Отдельно здесь важна стратегия с бакетами длины. В выбранной версии генерация распределяет сто попыток между разными длинами относительно старого пароля: исходная длина, длина меньше, длина больше, и несколько вариантов увеличения. Это сделано потому, что в таких данных новый пароль часто получается не полностью с нуля, а через добавление суффикса, префикса или небольшое изменение длины.

Также в `evaluator`-е есть защита от Out Of Memory. Если генерация падает из-за нехватки GPU-памяти, батч делится на меньшие части, CUDA-кэш очищается, и генерация повторяется. Для ноутбука с ограниченной VRAM это важно, потому что `OpenEvolve` вызывает `evaluator` много раз.

Важные параметры:

- `PASSLLM_GUESS_ONE_VERSION=4` — выбранная стратегия генерации;
- `guess_one_length_buckets` — распределение бюджета генераций по длинам;
- `PASSLLM_EXTRA_GUESSES_RATIO=0.2` — дополнительная генерация сверх лимита для отбора уникальных кандидатов;
- `PASSLLM_GPU_LOG_EVERY_N_REQUESTS=100` — как часто логировать состояние GPU;
- `EnableDump=500` — включение дампа промежуточных результатов;
- `max_length_sysprompt` — ограничение длины системного промпта.

11. **Кодовая ячейка 11. Подготовка системных пакетов для Ollama.** Одиннадцатая ячейка ещё раз подготавливает системное окружение под Ollama. Она устанавливает `zstd`, а также проверяет, есть ли `Linux`, `apt-get` и `nvidia-smi`. Если окружение похоже на Linux-машину с NVIDIA GPU, дополнительно ставятся `pciutils` и `lshw`.

Эти пакеты нужны не для самой логики генерации паролей, а для нормальной установки и диагностики Ollama. Проще поставить их заранее, чем потом получать ошибку установщика из-за отсутствующей системной утилиты.

12. **Кодовая ячейка 12. Установка, запуск Ollama и загрузка qwen3:8b.** Двенадцатая ячейка устанавливает Ollama, если бинарь `ollama` ещё не найден. После этого она запускает `ollama serve` в отдельном `daemon`-потоме и скачивает модель `qwen3:8b` командой `ollama pull qwen3:8b`.

Далее ячейка ждёт, пока локальный API станет доступен по адресу `http://127.0.0.1:11434/v1/models`. Это нужно, чтобы `OpenEvolve` мог обращаться к `qwen3:8b` как к локальной `OpenAI-compatible` модели. То есть эта ячейка фактически поднимает backend, который будет переписывать промпты.

Важные действия:

- установка Ollama через shell-скрипт, если она отсутствует;
- запуск `ollama serve` в фоне;

- загрузка qwen3:8b;
- проверка готовности локального API.

13. Кодовая ячейка 13. Патчи OpenEvolve и запуск эволюции. Тринадцатая ячейка запускает основной эксперимент. Сначала подключается `nest_asyncio`, чтобы асинхронный код OpenEvolve мог нормально работать внутри Jupyter notebook. Потом применяются патчи из `openevolve_notebook_patch`: жёсткий timeout для evaluator-a, очистка зависших дочерних Python-процессов и диагностика процессов после завершения. После подготовки ячейка загружает YAML-конфиг, созданный раньше, и вызывает `_run_evolution_async`. На вход передаются начальный промпт `initial_prompt.txt`, путь к evaluator-у, число итераций и папка для результатов. В итоге в переменной `result` остаётся лучший найденный промпт, его score и дополнительные метрики.

Важные параметры запуска:

- `initial_program=initial_prompt_path` — стартовый промпт;
- `evaluator=evaluator_path` — evaluator для оценки качества;
- `iterations=60` — число итераций эволюции;
- `output_dir=openevolve_output` — папка результатов;
- `cleanup=False` — рабочие артефакты не удаляются после запуска.

14. Кодовая ячейка 14. Сохранение лучшего промпта. Четырнадцатая ячейка сохраняет лучший промпт, который вернул OpenEvolve. Если в `result.best_code` есть текст, он записывается в файл `best_program.txt` внутри рабочей папки эксперимента.

Это удобный переход от этапа оптимизации к этапу генерации `submission`: сначала OpenEvolve нашёл хороший промпт, потом этот промпт сохраняется отдельно и дальше используется уже как финальный инструмент для генерации паролей на test-данных.

15. Кодовая ячейка 15. Генерация submission.csv. Пятнадцатая ячейка генерирует итоговый `submission.csv`. Она читает `test.json`, загружает `best_program.txt`, импортирует evaluator и для каждого тестового объекта генерирует по `NUM_SUBMISSION_GUESSES=100` кандидатов.

В этой ячейке есть важная практическая логика продолжения работы. Если `submission.csv` уже существует, код проверяет, какие строки уже заполнены, и не пересчитывает их повторно. Это полезно, потому что генерация может быть долгой: если ноутбук прервался, можно продолжить с места остановки.

Перед записью строки нормализуются. Пустые значения и похожие на null значения заменяются на технический fallback, чтобы в CSV не осталось пустых ячеек. Это не делает модель лучше, но помогает получить корректный файл для отправки.

Важные параметры:

- `NUM_SUBMISSION_GUESSES=100` — число кандидатов на один test-объект;
- `PROGRESS_EVERY=20` — как часто печатать прогресс;
- `TEST_PATH` — путь к `test.json`;

- `SUBMISSION_CSV_PATH` — путь к итоговому CSV;
- `min_new_tokens=4` и `max_new_tokens=20` — ограничения длины генерации в `submission`-режиме.

16. Кодовая ячейка 16. Экспорт паролей из CSV в TXT. Шестнадцатая ячейка переводит `submission.csv` в обычный текстовый файл `submission_passwords.txt`, где каждый пароль записан на отдельной строке. Колонка `id` пропускается, а пустые значения, нули, `null`, `none` и `nan` отбрасываются.

Это нужно не для Kaggle `submission`, а для удобной проверки. TXT-файл проще просматривать глазами, искать в нём повторяющиеся строки, проверять частоты или использовать во внешних скриптах.

17. Кодовая ячейка 17. Тестирование параметров `model.generate`. Семнадцатая ячейка нужна для тестирования параметров `model.generate`. Она генерирует для разных вариантов `guess_one` по `N_GUESSES=100` паролей для подвыборки старых паролей. В коде задано `N_OLD_PASSWORDS=100`, то есть берётся сто старых паролей для диагностики.

Здесь сравниваются разные версии `guess_one` и разные распределения `guess_one_length_buckets`. Идея простая: посмотреть, какие настройки длины и генерации дают лучший `cracked_rate`. Для каждого варианта сохраняется JSON-дамп с исходным старым паролем, правильным новым паролем, списком кандидатов и признаком попадания.

Эта ячейка не является обязательной для финального `submission`, но она полезна как исследовательский блок. По ней можно понять, почему выбран именно такой вариант бакетов и почему часть бюджета генерации отдаётся паролям большей длины.

Важные параметры:

- `N_GUESSES=100` — число кандидатов на один пример;
- `N_OLD_PASSWORDS=100` — размер диагностической подвыборки;
- `ALL_GUESS_ONE_VERSIONS` — список тестируемых стратегий генерации;
- `ALL_GUESS_ONE_LENGTH_BUCKETS` — наборы распределений по длинам;
- `PASSLLM_EXTRA_GUESSES_RATIO=0.0` — отключение дополнительных генераций для чистого сравнения.

18. Кодовая ячейка 18. Очистка PyTorch cache и памяти ускорителя. Восемнадцатая ячейка очищает память после тяжёлых запусков. Она ищет загруженные `evaluator`-модули, сбрасывает кэшированные `_cached_model` и `_cached_tokenizer`, удаляет глобальные переменные `model`, `tokenizer` и `evaluator`, а потом запускает `gc.collect()`.

Если доступна CUDA, ячейка синхронизирует GPU, очищает CUDA cache, собирает IPC-объекты, сбрасывает `peak memory stats` и печатает свободную и общую VRAM. Если используется Apple MPS, очищается MPS cache. Эта ячейка нужна, чтобы после экспериментов можно было продолжать работу в том же `notebook kernel` без полной перезагрузки окружения.

Важные действия:

- сброс кэшированных модели и токенизатора;
- удаление тяжёлых глобальных переменных;

- `torch.cuda.empty_cache()` — освобождение CUDA cache;
- `torch.cuda.mem_get_info()` — печать свободной и общей GPU-памяти.

Итоговое замечание

В итоге ноутбук устроен как последовательный pipeline: сначала ставятся зависимости и настраиваются пути, затем поднимаются backend-и для генерации и оптимизации, потом OpenEvolve ищет лучший промпт, после этого этот промпт используется для создания submission, а последние ячейки помогают проверить параметры генерации и освободить память. Такая структура удобна для преподавателя и кандидата по ИИ, потому что видно не только что запускается, но и зачем каждая ячейка нужна в общем процессе.